```
!pip install torch torchvision pillow matplotlib gradio -q

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms
import torchvision.models as models
from PIL import Image, ImageFile
import os
import matplotlib.pyplot as plt
import gradio as gr
import csv
import torch.nn.functional as F

ImageFile.LOAD_TRUNCATED_IMAGES = True

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
Using device: cuda
```

```
transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(256),
    transforms.ToTensor()
])

def load_images(folder):
    images = []
    for file in os.listdir(folder):
        if file.lower().endswith((".jpg", ".png", ".jpeg")):
            path = os.path.join(folder, file)
            try:
                img = Image.open(path).convert("RGB")
                images.append(transform(img))
            except:
                print(f"Skipping corrupted image: {file}")
    return images

content_images = load_images("data/content")
style_images = load_images("data/style")

print("Content:", len(content_images))
print("Style:", len(style_images))
```

```
Content: 48
Style: 30
```

```
vgg = models.vgg19(weights=models.VGG19_Weights.DEFAULT).features[:21].to(device).eval()

for param in vgg.parameters():
    param.requires_grad = False
```

```
Downloading: "https://download.pytorch.org/models/vgg19-dcbb9e9d.pth" to /root/.cache/torch/hub/checkpoints/vgg19-dcbb9e9d.pth
100%|██████████| 548M/548M [00:05<00:00, 103MB/s]
```

```
class SANet(nn.Module):
    def __init__(self, in_channels):
        super().__init__()
        self.f = nn.Conv2d(in_channels, in_channels, 1)
        self.g = nn.Conv2d(in_channels, in_channels, 1)
        self.h = nn.Conv2d(in_channels, in_channels, 1)
        self.out_conv = nn.Conv2d(in_channels, in_channels, 1)
        self.softmax = nn.Softmax(dim=-1)

    def forward(self, content, style):
        B, C, H, W = content.size()

        f_content = self.f(content).view(B, C, -1)
        g_style = self.g(style).view(B, C, -1)
        h_style = self.h(style).view(B, C, -1)

        attention = torch.bmm(f_content.permute(0, 2, 1), g_style)
        attention = self.softmax(attention)
```

```
        out = torch.bmm(h_style, attention.permute(0, 2, 1))
        out = out.view(B, C, H, W)
        out = self.out_conv(out)

        return out + content
```

```
class Decoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Conv2d(512, 256, 3, padding=1),
            nn.ReLU(),
            nn.Upsample(scale_factor=2),

            nn.Conv2d(256, 256, 3, padding=1),
            nn.ReLU(),

            nn.Conv2d(256, 128, 3, padding=1),
            nn.ReLU(),
            nn.Upsample(scale_factor=2),

            nn.Conv2d(128, 128, 3, padding=1),
            nn.ReLU(),

            nn.Conv2d(128, 64, 3, padding=1),
            nn.ReLU(),
            nn.Upsample(scale_factor=2),

            nn.Conv2d(64, 3, 3, padding=1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.model(x)
```

```
sanet = SANet(512).to(device)
decoder = Decoder().to(device)

optimizer = optim.Adam(
    list(sanet.parameters()) + list(decoder.parameters()),
    lr=1e-4
)
```

```
def psnr(img1, img2):
    mse = F.mse_loss(img1, img2)
    if mse == 0:
        return 100
    return 20 * torch.log10(1.0 / torch.sqrt(mse))

def ssim(img1, img2):
    C1 = 0.01 ** 2
    C2 = 0.03 ** 2

    mu1 = img1.mean()
    mu2 = img2.mean()

    sigma1 = ((img1 - mu1) ** 2).mean()
    sigma2 = ((img2 - mu2) ** 2).mean()
    sigma12 = ((img1 - mu1) * (img2 - mu2)).mean()

    return ((2 * mu1 * mu2 + C1) * (2 * sigma12 + C2)) / \
           ((mu1**2 + mu2**2 + C1) * (sigma1 + sigma2 + C2))
```

```
EPOCHS = 30

def mean_std(x):
    mean = x.mean([2,3], keepdim=True)
    std = x.std([2,3], keepdim=True)
    return mean, std

with open("training_metrics.csv", mode="w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Epoch", "Total Loss", "Content Loss", "Style Loss", "PSNR", "SSIM"])
```

```
    for epoch in range(EPOCHS):
        total_loss = 0
        total_content_loss = 0
        total_style_loss = 0
        total_psnr = 0
        total_ssim = 0

        for content_img in content_images:
            for style_img in style_images:

                content = content_img.unsqueeze(0).to(device)
                style = style_img.unsqueeze(0).to(device)

                content_feat = vgg(content)
                style_feat = vgg(style)

                t = sanet(content_feat, style_feat)
                output = decoder(t)

                output_feat = vgg(output)

                content_loss = torch.mean((output_feat - content_feat)**2)

                s_mean, s_std = mean_std(style_feat)
                o_mean, o_std = mean_std(output_feat)

                style_loss = torch.mean((o_mean - s_mean)**2 + (o_std - s_std)**2)

                loss = content_loss + 10 * style_loss

                optimizer.zero_grad()
                loss.backward()
                optimizer.step()

                # Metrics
                psnr_val = psnr(output, content)
                ssim_val = ssim(output, content)

                total_loss += loss.item()
                total_content_loss += content_loss.item()
                total_style_loss += style_loss.item()
                total_psnr += psnr_val.item()
                total_ssim += ssim_val.item()

        num_steps = len(content_images) * len(style_images)

        avg_total = total_loss / num_steps
        avg_content = total_content_loss / num_steps
        avg_style = total_style_loss / num_steps
        avg_psnr = total_psnr / num_steps
        avg_ssim = total_ssim / num_steps

        print(f"Epoch {epoch+1}/{EPOCHS}")
        print(f"Total Loss: {avg_total:.4f}")
        print(f"Content Loss: {avg_content:.4f}")
        print(f"Style Loss: {avg_style:.4f}")
        print(f"PSNR: {avg_psnr:.2f} | SSIM: {avg_ssim:.4f}")
        print("-"*50)

        writer.writerow([epoch+1, avg_total, avg_content, avg_style, avg_psnr, avg_ssim])

torch.save({
    "sanet": sanet.state_dict(),
    "decoder": decoder.state_dict()
}, "sanet_model.pth")

print("Model + CSV saved!")
```

```
Epoch 1/30
Total Loss: 9.8932
Content Loss: 4.9347
Style Loss: 0.4958
PSNR: 10.78 | SSIM: 0.1278
--------------------------------------------------
Epoch 2/30
Total Loss: 6.8391
Content Loss: 4.0071
```

```
Style Loss: 0.2832
PSNR: 11.76 | SSIM: 0.2642
--------------------------------------------------
Epoch 3/30
Total Loss: 5.9009
Content Loss: 3.5939
Style Loss: 0.2307
PSNR: 11.98 | SSIM: 0.3101
--------------------------------------------------
Epoch 4/30
Total Loss: 5.3832
Content Loss: 3.3410
Style Loss: 0.2042
PSNR: 12.07 | SSIM: 0.3254
--------------------------------------------------
Epoch 5/30
Total Loss: 4.9196
Content Loss: 3.1420
Style Loss: 0.1778
PSNR: 12.17 | SSIM: 0.3418
--------------------------------------------------
Epoch 6/30
Total Loss: 4.6568
Content Loss: 2.9974
Style Loss: 0.1659
PSNR: 12.27 | SSIM: 0.3541
--------------------------------------------------
Epoch 7/30
Total Loss: 4.4315
Content Loss: 2.8852
Style Loss: 0.1546
PSNR: 12.32 | SSIM: 0.3598
--------------------------------------------------
Epoch 8/30
Total Loss: 4.2495
Content Loss: 2.7904
Style Loss: 0.1459
PSNR: 12.37 | SSIM: 0.3673
--------------------------------------------------
Epoch 9/30
Total Loss: 4.0708
Content Loss: 2.7081
Style Loss: 0.1363
PSNR: 12.41 | SSIM: 0.3737
--------------------------------------------------
Epoch 10/30
Total Loss: 3.9760
Content Loss: 2.6452
Style Loss: 0.1331
```

```
from google.colab import files
files.download("training_metrics.csv")
```

```
def show(img):
    img = img.squeeze().permute(1,2,0).cpu().detach().numpy()
    plt.imshow(img)
    plt.axis("off")

from google.colab import files

print("Upload Content Image")
c_file = files.upload()
c_path = list(c_file.keys())[0]

print("Upload Style Image")
s_file = files.upload()
s_path = list(s_file.keys())[0]

content = transform(Image.open(c_path)).unsqueeze(0).to(device)
style = transform(Image.open(s_path)).unsqueeze(0).to(device)

with torch.no_grad():
    t = sanet(vgg(content), vgg(style))
    output = decoder(t)

print("PSNR:", psnr(output, content).item())
print("SSIM:", ssim(output, content).item())

plt.figure(figsize=(10,4))
```

```
plt.subplot(1,2,1); show(content); plt.title("Content")
plt.subplot(1,2,2); show(output); plt.title("Output")
plt.show()
```

```
Upload Content Image
```

Choose Files golden_gate.jpg

golden_gate.jpg(image/jpeg) - 102100 bytes, last modified: 3/6/2026 - 100% done

```
Saving golden_gate.jpg to golden_gate.jpg
Upload Style Image
```

Choose Files starry_night.jpg

starry_night.jpg(image/jpeg) - 315236 bytes, last modified: 3/6/2026 - 100% done

```
Saving starry_night.jpg to starry_night.jpg
PSNR: 11.163591384887695
SSIM: 0.3395400643348694
```